
Python 金融数据分析

第 8 章 数学工具

主讲人： 王伟冠, 吕文俊
院 系： 经济学院金融系

上海大学

2026 年 9 月 24 日



1. 金融为什么需要数学工具
2. 函数逼近：回归
3. 函数逼近：插值
4. 优化：寻找最优解
5. 积分：从面积到期权价格
6. 符号计算：SymPy
7. 小结与练习



1. 金融为什么需要数学工具

2. 函数逼近：回归

3. 函数逼近：插值

4. 优化：寻找最优解

5. 积分：从面积到期权价格

6. 符号计算：SymPy

7. 小结与练习



- 会用**回归** (`np.polyfit`) 与**插值** (样条) 逼近函数与数据
- 会用 `scipy.optimize` 做全局搜索、局部优化与**带约束优化**
- 理解数值积分与蒙特卡洛积分, 知道它们与期权定价的关系
- 初步掌握 SymPy 符号计算: 解方程、求导、积分

四大工具与金融应用

逼近 (回归/插值)	收益率曲线补全、美式期权估值
凸优化	模型校准、投资组合选择
积分	衍生品定价 (期望收益的折现)
符号计算	推导解析公式 (如希腊字母)

- 教材用一个“正弦 + 线性”函数贯穿全章：

$$f(x) = \sin(x) + 0.5x$$

- 它既有弯曲部分（考验拟合能力）又有趋势项，很适合演示逼近方法。
- 第 6 章我们用 `np.polyfit` 估计过 `beta`——那就是回归的金融应用；本章把它推广为通用的函数逼近工具。

零基础提醒

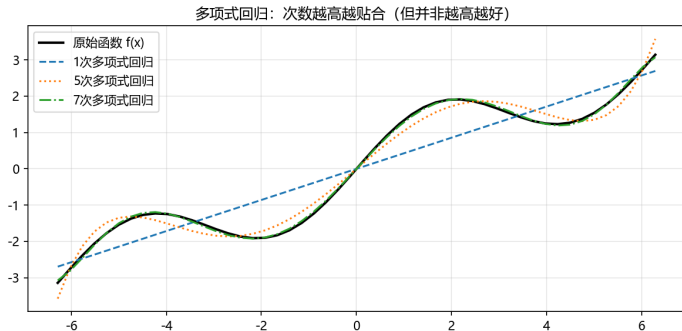
数学符号不必害怕：本章重点是“会调用、会读图、会解释”，推导细节留给专门课程。

1. 金融为什么需要数学工具
- 2. 函数逼近：回归**
3. 函数逼近：插值
4. 优化：寻找最优解
5. 积分：从面积到期权价格
6. 符号计算：SymPy
7. 小结与练习



```
1 import numpy as np
2
3 def f(x):
4     return np.sin(x) + 0.5 * x
5
6 x = np.linspace(-2 * np.pi, 2 * np.pi, 50)
7 reg = np.polyfit(x, f(x), deg=7) # 7次多项式拟合
8 ry = np.polyval(reg, x)         # 用拟合结果求值
9 np.mean((f(x) - ry) ** 2)       # 均方误差 MSE 0.0018
```

- `np.polyfit(x, y, deg)`: 找一条 `deg` 次多项式, 使它与数据点的误差平方和最小 (最小二乘)。
- `np.polyval(reg, x)`: 把拟合系数代入求值。
- MSE 越小拟合越好: 1 次 ≈ 0.42 , 5 次 ≈ 0.054 , 7 次 ≈ 0.0018 。



- **抗噪声**：数据带噪声时，回归曲线反而比数据点更接近真实函数——它在“平均掉”噪声。
- **不要求排序**：数据点顺序打乱也不影响结果。

过拟合警告

多项式次数过高时，曲线会穿过每个噪声点、在点之间剧烈震荡——对样本“背得滚瓜烂熟”，对新数据一塌糊涂。次数选择要配合样本外检验（后面的机器学习思想）。

1. 金融为什么需要数学工具
2. 函数逼近：回归
- 3. 函数逼近：插值**
4. 优化：寻找最优解
5. 积分：从面积到期权价格
6. 符号计算：SymPy
7. 小结与练习



```
1 import scipy.interpolate as spi
2
3 xa = np.linspace(-2 * np.pi, 2 * np.pi, 12) # 12个观测点
4 ya = f(xa)
5 ipo = spi.splrep(xa, ya, k=3) # 构造三次样条
6 xs = np.linspace(-2 * np.pi, 2 * np.pi, 200)
7 iy = spi.splev(xs, ipo) # 在更密的网格上求值
```

- splrep (构造) + splev (求值), 与 polyfit+polyval 的分工一模一样。
- k=3: 三次样条——分段三次多项式, 连接处平滑可导。

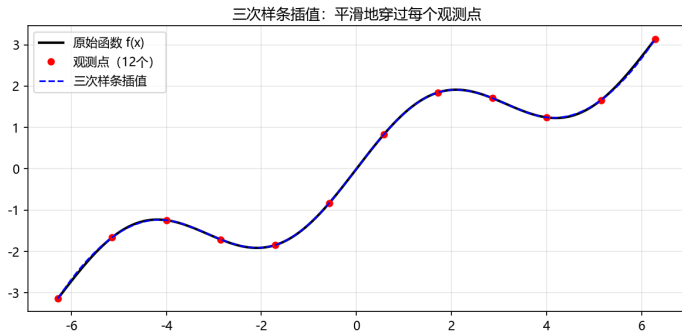


图 2: 只用 12 个点, 三次样条就能平滑还原整条曲线

- 市场上只有整期限的国债利率：1 年、3 年、5 年、10 年...
- 定价需要 7.5 年的利率怎么办？——插值。

```
1 import pandas as pd
2 maturity = [1, 3, 5, 10]           # 已知期限（年）
3 rate      = [1.5, 2.1, 2.6, 3.2]   # 对应利率（%）
4 ipo = spi.splrep(maturity, rate, k=3)
5 spi.splev(7.5, ipo).round(3)       # 7.5年期利率  3.04%
```

插值的限制

要求数据**有序且无噪声**，高维问题不适用——这正是回归的主场。

1. 金融为什么需要数学工具
2. 函数逼近：回归
3. 函数逼近：插值
- 4. 优化：寻找最优解**
5. 积分：从面积到期权价格
6. 符号计算：SymPy
7. 小结与练习



- 目标函数有多个“坑”（局部极小值），全局最小值藏在其中：

目标函数曲面（多个局部极小）与全局最小点

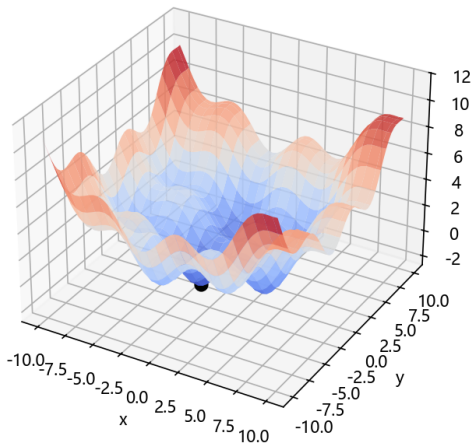


图 3: $f(x, y) = \sin x + 0.05x^2 + \sin y + 0.05y^2$, 黑点为全局最小

```

1 import scipy.optimize as sco
2
3 def fm(p):
4     x, y = p
5     return np.sin(x) + 0.05 * x ** 2 \
6         + np.sin(y) + 0.05 * y ** 2
7
8 # 1. 全局：网格上逐点尝试（暴力但可靠）
9 opt1 = sco.brute(fm, ((-10, 10.1, 0.1), (-10, 10.1, 0.1)),
10                 finish=None)      # 找到约 (-1.4, -1.4)
11 # 2. 局部：从粗搜结果出发精细下降
12 opt2 = sco.fmin(fm, opt1, xtol=0.001, ftol=0.001, disp=False)
13 opt2.round(4)      # [-1.4274, -1.4279], f = -1.7757

```

- 局部算法 (fmin) 像“闭眼下山”：从哪出发，就掉进哪个坑。
- 从 (2, 2) 出发只能得到 $f \approx 0.02$ 的“局部最优”，错过真正的全局最小 -1.78。
- brute 先在网格上扫一遍，锁定最深的坑，再交给 fmin 精修。

金融中的教训

模型校准（如用市场报价反推期权模型参数）是典型的优化问题；不加全局搜索，可能校准到一个“局部最优”的错误参数组合。

- 预算 100 元买两种证券 (单价均 10 元); 一年后状态 u/d 等概率, 收益向量分别为 $(15, 5)$ 与 $(5, 12)$; 效用 $u(w) = \sqrt{w}$ 。求最大化期望效用的购买数量。

```
1 import math
2 def eu(p):          # 最小化 "负效用" = 最大化效用
3     s, b = p
4     return -(0.5 * math.sqrt(s * 15 + b * 5)
5             + 0.5 * math.sqrt(s * 5 + b * 12))
6 cons = [{"type": "ineq", "fun":
7         lambda p: 100 - p[0] * 10 - p[1] * 10}] # 预算约束
8 res = sco.minimize(eu, (5, 5), method="SLSQP",
9                   bounds=((0, 10), (0, 10)),
10                  constraints=cons)
11 res.x.round(2)     # [8.03, 1.97]: 花光全部预算
```

1. 金融为什么需要数学工具
2. 函数逼近：回归
3. 函数逼近：插值
4. 优化：寻找最优解
- 5. 积分：从面积到期权价格**
6. 符号计算：SymPy
7. 小结与练习



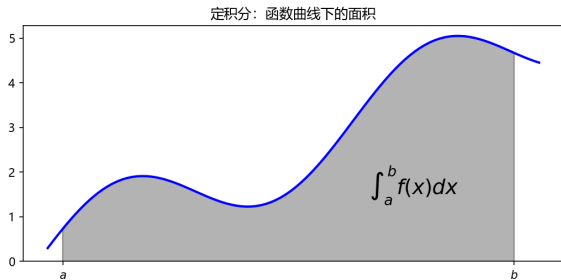


图 4: $\int_{0.5}^{9.5} f(x) dx$: 衍生品定价的数学本质就是这类计算



```
1 import scipy.integrate as sci
2
3 a, b = 0.5, 9.5
4 sci.quad(f, a, b)[0]      # 24.37475... (自适应求积, 最常用)
```

- `quad` 返回 (结果, 误差估计), 通常取第一个元素。
- 数据点形式则用 `sci.trapz` (梯形法则)。
- **金融意义**: 期权在风险中性测度下的价值 = 期望折现收益; 连续情形下, 期望就是一个积分。

```
1 rng = np.random.default_rng(1000)
2 N = 500000
3 x = rng.random(N) * (b - a) + a    # 区间内均匀撒点
4 np.mean(f(x)) * (b - a)            # 平均函数值 × 区间长度
5 # 结果 24.36, 与 quad 基本一致
```

- 原理： $\int_a^b f(x)dx = (b - a) \cdot E[f(x)]$ ，用样本均值逼近期望。
- 样本越多越准，但**收敛不单调**，误差按 $1/\sqrt{N}$ 下降。

为什么金融偏爱蒙特卡洛？

高维（多资产、多期）积分数值方法会“维度爆炸”，蒙特卡洛的收敛速度却与维度无关——它是复杂衍生品定价的主力工具（下下章的主题）。

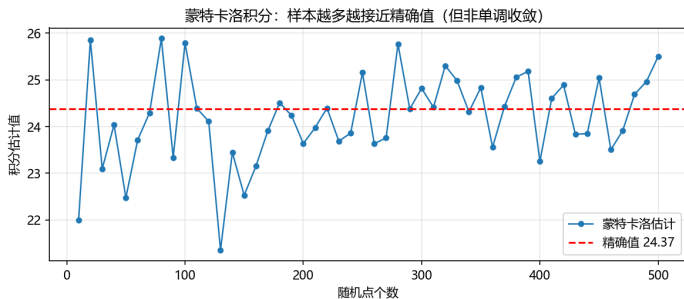


图 5: 估计值围绕精确值（红色虚线）震荡收窄

1. 金融为什么需要数学工具
2. 函数逼近：回归
3. 函数逼近：插值
4. 优化：寻找最优解
5. 积分：从面积到期权价格
- 6. 符号计算：SymPy**
7. 小结与练习



```
1 import sympy as sy
2
3 x = sy.Symbol("x")          # x 是符号，不是数字
4 f = sy.sin(x) + 0.5 * x     # 符号表达式
5 sy.solve(x ** 2 - 1, x)     # 解方程：[-1, 1]
6 sy.simplify((x ** 2 - 1) / (x - 1))  # 化简：x + 1
```

- NumPy 算的是“数”，SymPy 算的是“式子”。
- 在 Notebook 中，SymPy 表达式会以漂亮的数学公式显示。

```
1 sy.diff(f, x)           # 求导:  $\cos(x) + 0.5$ 
2 sy.integrate(f, x)      # 不定积分:  $0.25x^2 - \cos(x)$ 
3 sy.integrate(f, (x, 0.5, 9.5)).evalf() # 定积分: 24.3748
```

- 与数值积分 quad 的结果完全一致——互相验证的好例子。
- 金融应用：期权价格对参数的导数（Delta、Gamma 等“希腊字母”）可用符号求导得到解析式。

数值 vs 符号

要“精确公式”用 SymPy；要“大规模快速计算”用 NumPy/SciPy。两者互补。

1. 金融为什么需要数学工具
2. 函数逼近：回归
3. 函数逼近：插值
4. 优化：寻找最优解
5. 积分：从面积到期权价格
6. 符号计算：SymPy
- 7. 小结与练习**



- **回归**: `polyfit/polyval`, 抗噪声、不要求排序; 警惕过拟合。
- **插值**: `splrep/splev`, 有序无噪声数据的平滑还原; 典型应用是收益率曲线。
- **优化**: `brute` (全局粗搜) $\rightarrow$ `fmin` (局部精修); 带约束用 `minimize + constraints`。
- **积分**: `quad` 精确快速; 蒙特卡洛 $\bar{f} \times (b - a)$ 通吃高维问题。
- **符号**: SymPy 解方程、求导、积分, 推导解析公式。



课后任务

- ① 对 $g(x) = \cos(x) + 0.2x$ 分别做 3 次与 9 次多项式回归，比较 MSE 并绘图。
- ② 给定期限 $[1, 2, 5, 10]$ 年、利率 $[1.6, 1.9, 2.5, 3.1]$ ，用三次样条插值估计 3.5 年期利率。
- ③ 选做：把蒙特卡洛积分的样本数从 10^3 增加到 10^6 ，记录误差变化，验证 $1/\sqrt{N}$ 规律。

下章预告

数学工具让代码表达力倍增，但循环写多了会变慢。下一章讲**性能**：为什么向量化比循环快，以及如何科学地计时。

本章内容改编自教材第 11 章^[1]。

- [1] 伊夫·希尔皮斯科. Python 金融大数据分析 [M]. 2nd ed. 北京: 人民邮电出版社, 2018.



Thanks for your attention!

Q & A

